

Fix Report

Tổng quan

Quá trình phát triển từ bản triển khai ban đầu đến phiên bản mới nhất đã biến đổi một hệ thống Hybrid FTP cơ bản (Kênh điều khiển TCP + Kênh dữ liệu UDP RDT) thành một bản triển khai mạnh mẽ, an toàn và tuân thủ hoàn toàn các tiêu chuẩn FTP.

Dưới đây là báo cáo chi tiết về các thay đổi được thực hiện đối với cả mã nguồn Client (Máy khách) và Server (Máy chủ), cũng như các vấn đề, lỗi và lỗ hổng bảo mật cụ thể mà những thay đổi này đã giải quyết.

Các thay đổi ở phía Client & Vấn đề đã được giải quyết

1. Khắc phục lỗi tự động xóa tệp đã tải xuống

- **Thay đổi:** Trong client gốc (`rdt_download`), biến `success` được khởi tạo là `False` và không bao giờ được cập nhật thành `True` khi hoàn tất tải tệp thành công. Ở client mới nhất, `success = True` được thiết lập khi nhận được gói tin `FLAG_FIN`.
- **Vấn đề đã giải quyết:** Trong phiên bản cũ, khối `finally` luôn đánh giá `if not success` là `True`, dẫn đến việc tự động xóa ngay lập tức mọi tệp dù đã được tải xuống thành công. Bản cập nhật đảm bảo các tệp đã tải xuống được giữ lại nguyên vẹn.

2. Hỗ trợ đầy đủ Chế độ Chủ động (PORT)

- **Thay đổi:** Đã thêm tính năng theo dõi `active_mode`, liên kết (bind) socket trên `0.0.0.0:<data_port>`, và tham số `is_active_mode` trong `rdt_upload`. Khi tải lên ở Chế độ Chủ động (Active Mode), client sẽ đợi gói tin UDP ping/SYN ban đầu từ máy chủ trước khi bắt đầu truyền dữ liệu.
- **Vấn đề đã giải quyết:** Client cũ chỉ hỗ trợ Chế độ Bị động (PASV) thông qua các lệnh macro (`GET`, `PUT`, `LIST`). Client được cập nhật nay cho phép đàm phán `PORT` tiêu chuẩn cho các kênh dữ liệu ở Chế độ Chủ động.

3. Xử lý ngắt từ người dùng & Hủy bỏ bất đồng bộ (ABOR)

- **Thay đổi:** Đã thêm các khối `try...except KeyboardInterrupt` bên trong `rdt_download` và `rdt_upload`. Khi người dùng nhấn `Ctrl+C` trong quá trình truyền dữ liệu, client sẽ bắt ngoại lệ và ngay lập tức truyền lệnh `ABOR` tới máy chủ qua socket điều khiển TCP.
- **Vấn đề đã giải quyết:** Trước đây, việc ngắt quá trình truyền bằng `Ctrl+C` sẽ làm ngưng trệ (crash) hoàn toàn tiến trình client cục bộ, đồng thời khiến máy chủ phải chờ đợi các gói tin UDP vô thời hạn hoặc cho đến khi bị hết thời gian chờ (timeout).

4. Xử lý trực tiếp các lệnh FTP tiêu chuẩn trong CLI

- **Thay đổi:** Đã mở rộng vòng lặp CLI (giao diện dòng lệnh) chính để xử lý trực tiếp các lệnh FTP tiêu chuẩn (`RETR`, `STOR`, `APPE`, `STOU`, `NLST`, `PASV`, `PORT`) song song với các macro phím tắt (`GET`, `PUT`, `LIST`).
- **Vấn đề đã giải quyết:** Trong phiên bản cũ, việc nhập các lệnh FTP tiêu chuẩn như `RETR file.txt` sẽ gửi yêu cầu TCP tới máy chủ, nhưng client không bao giờ khởi chạy

các quy trình nhận hoặc gửi dữ liệu UDP RDT cục bộ (`rdt_download/rdt_upload`), dẫn đến việc kênh dữ liệu bị treo.

Các thay đổi ở phía Server & Vấn đề đã được giải quyết

1. Cô lập môi trường (Sandboxing) & Ngăn chặn lỗ hổng Path Traversal

- **Thay đổi:** Giới thiệu cơ chế `self.base_dir = os.path.abspath(os.getcwd())` và thêm các bước kiểm tra xác minh quyền truy cập (`if not target_path.startswith(self.base_dir):`) trên tất cả các lệnh xử lý thư mục và tệp (`CWD`, `MKD`, `RMD`, `LIST`, `NLST`, `RETR`, `STOR`, `DELE`, `RNFR`, `RNTO`, `APPE`, `MDTM`).
- **Vấn đề đã giải quyết:** Server cũ đã sử dụng `os.path.abspath` mà không kiểm tra xem đường dẫn đích có vi phạm (thoát ra khỏi) thư mục gốc của máy chủ hay không. Trước đây, người dùng có thể thực thi các lệnh như `CDUP` hoặc truyền vào các đường dẫn `../.` để thoát khỏi thư mục bị hạn chế và truy cập trái phép vào các tệp hệ thống.

2. Hủy bỏ bất đồng bộ không chặn (Non-Blocking) qua Socket Multiplexing (`select.select`)

- **Thay đổi:** Đã thay thế các lệnh gọi socket `recvfrom` đơn thuần trong `rdt_send` và `rdt_recv` bằng phương pháp ghép kênh I/O (I/O multiplexing): `select.select([self.control_sock, self.data_sock], [], [], timeout)`.
- **Vấn đề đã giải quyết:** Trước đây, trong khi truyền dữ liệu UDP, luồng (thread) của máy chủ bị chặn lại do *chỉ* lắng nghe trên socket UDP. Nếu client gửi lệnh `ABOR` qua TCP, máy chủ sẽ bỏ qua lệnh này cho đến khi toàn bộ quá trình truyền UDP kết thúc hoặc hết thời gian chờ. Nhờ `select`, máy chủ hiện có thể giám sát cả hai socket đồng thời và hủy luồng truyền UDP ngay lập tức khi nhận được lệnh `ABOR`.

3. Khởi tạo kết nối UDP ở Chế độ Chủ động

- **Thay đổi:** Trong hàm `handle_port`, IP và Cổng (Port) của client được lưu trữ dưới dạng biến `self.client_data_ip` và `self.client_data_port`. Trong quá trình tải lên ở Chế độ Chủ động (`STOR`, `APPE`, `STOU`), máy chủ sẽ bắn một gói tin UDP `FLAG_SYN` ban đầu tới socket UDP đang lắng nghe của client trước khi gọi `rdt_recv`.
- **Vấn đề đã giải quyết:** Giải quyết vấn đề vòng lặp "con gà và quả trứng" trong kết nối, nơi client đợi tín hiệu UDP ping của máy chủ trong Chế độ Chủ động, trong khi máy chủ cũng đồng thời đợi dữ liệu của client mà không hề chủ động liên lạc trước.

4. Xác thực trạng thái sẵn sàng của Data Socket

- **Thay đổi:** Bổ sung các bước kiểm tra trước khi truyền (`if not self.data_sock:` `self.send_response("425 Use PORT or PASV first.\r\n")`) vào tất cả các lệnh cần truyền dữ liệu (`RETR`, `STOR`, `APPE`, `STOU`, `LIST`, `NLST`).
- **Vấn đề đã giải quyết:** Ngăn chặn các ngoại lệ hoặc rủi ro sập (crash) hệ thống không mong muốn trên máy chủ nếu client cố gắng bắt đầu truyền dữ liệu mà chưa thiết lập `PORT` hoặc `PASV` từ trước.

5. Dọn dẹp trạng thái đổi tên (Rename) & Cải tiến định dạng

- **Thay đổi:**

- Đặt lại `self.rename_from_path = None` bất cứ khi nào có một lệnh khác ngoài `RNT0` được ban hành.
- Cải thiện đầu ra của `handle_list` để bao gồm các chuỗi quyền theo tiêu chuẩn hệ thống UNIX (`drwxr-xr-x`, `-rw-r--r--`) và dấu thời gian sửa đổi (`datetime.datetime.fromtimestamp(mtime).strftime('%b %d %H:%M')`).
- **Vấn đề đã giải quyết:** Sửa lỗi trạng thái đổi tên bị treo nếu người dùng thực thi `RNFR` sau đó là một lệnh không liên quan; đồng thời cung cấp cho các ứng dụng máy khách danh sách tệp tuân thủ chuẩn định dạng UNIX.

Bảng Tóm tắt So sánh

Tính năng / Khu vực	Tệp cũ (Nguồn 14 & 15)	Tệp mới nhất (Nguồn 16 & 17)	Lợi ích chính / Vấn đề đã được giải quyết
Tệp đã tải xuống	Bị xóa khi hoàn tất do lỗi cờ <code>success</code> .	Được giữ lại chính xác khi nhận được <code>FLAG_FIN</code> .	Khắc phục lỗi xóa tệp sau khi tải xuống.
Bảo mật Thư mục	Cho phép Path Traversal qua <code>..</code> hoặc <code>CDUP</code> .	Bị giới hạn nghiêm ngặt trong hộp cát (sandbox) <code>base_dir</code> .	Ngăn chặn hành vi truy cập tệp trái phép.
Thao tác Hủy (Abort)	Bị chặn lại trong quá trình dòng UDP đang truyền; lệnh <code>ABOR</code> bị trì hoãn.	Ghép kênh socket không chặn (Non-blocking) qua hàm <code>select</code> .	Có thể hủy ngay lập tức các quá trình truyền dữ liệu đang diễn ra.
Chế độ Dữ liệu	Chỉ hỗ trợ chế độ Bị động (<code>PASV</code>).	Hỗ trợ đầy đủ chế độ Chủ động (<code>PORT</code>) & Bị động (<code>PASV</code>).	Đạt sự tuân thủ tiêu chuẩn giao thức FTP.

Xử lý CLI	Chỉ hỗ trợ các macro (GET, PUT, LIST).	Hỗ trợ các lệnh FTP thô (RETR, STOR, STOU, APPE, v.v.).	Cải thiện khả năng sử dụng của client & tuân thủ giao thức.
------------------	--	---	---